

(cansat and cubesat satellite)



INDIA SPACE LAB

Project Report

17th June to 31st July 2026

Student Name: PALAK J. JESHRANI

Institute Name: Ajeenkya DY Patil University

Institute Roll no.: E25I051050

Enrollment no.: ISL-571019

CanSat Ground Control Software (GCS)

Technical Implementation Report

Module	Telemetry Simulation & Live Dashboard Engine
Core Script	dashboard.js - telemetry, charts, GPS, mission control
Libraries Used	Chart.js, Leaflet.js (OpenStreetMap), MediaDevices API
Update Frequency	1 telemetry packet per second
Report Type	Technical Report

Table of Contents

- 1. Overview of the Script Architecture
- 2. Global State and Chart Data Structures
- 3. Chart.js Graph Configuration
- 4. Telemetry Simulation Engine
- 5. Dashboard Update Logic
- 6. Live Graph Update Mechanism
- 7. Error Code Generation System
- 8. Mission Command Functions
- 9. Data Export (CSV and Graph Images)
- 10. GPS Map Integration (Leaflet.js)
- 11. Live Camera Integration
- 12. Summary of Functions

Software Design and Implementation

1. Overview of the Script Architecture

The `main.js` file serves as the core controller of the CanSat Ground Control Software (GCS). It manages all dynamic operations of the application, including telemetry simulation, dashboard updates, graph visualization, mission command execution, GPS tracking, error monitoring, data export, and camera integration. The application follows an event-driven architecture where user interactions and simulated telemetry packets continuously update the interface.

The script begins by declaring global variables and initializing Chart.js graphs. Once the operator starts telemetry, the simulation engine generates telemetry packets every second. These packets are processed and distributed to different modules such as the telemetry dashboard, graphs, GPS tracker, packet log, and orientation visualization. The modular design improves readability, maintainability, and scalability while allowing future integration with real hardware using Web Serial API.

2. Global State and Chart Data Structures

Several global variables are declared at the beginning of the script to maintain the application's state throughout execution.

The `running` variable controls whether telemetry generation is active or stopped. The `telemetry` array stores all generated telemetry packets, allowing them to be exported as CSV files later. The `packetNumber` variable maintains the packet count, while the `stream` variable stores the browser camera stream.

Separate arrays are created for each telemetry parameter, including:

- Time Data
- Altitude Data
- Temperature Data
- Pressure Data
- Descent Rate Data
- Battery Voltage Data

These arrays continuously receive new telemetry values and act as the data source for the Chart.js graphs. To prevent excessive memory usage, older values are removed after a predefined number of samples, maintaining a smooth real-time visualization.

3. Chart.js Graph Configuration

The Ground Control Software uses Chart.js to visualize mission telemetry through five continuously updating line graphs.

The graphs implemented in the project are:

- Altitude vs Time
- Temperature vs Time
- Pressure vs Time
- Descent Rate vs Time
- Battery Voltage vs Time

Each graph is configured using a common configuration object to maintain consistency throughout the dashboard. The configuration includes:

- Disabled animation for smoother updates
- Graph titles
- Legends
- Time labels on the X-axis
- Sensor values on the Y-axis
- Continuous line plotting
- Automatic scaling

Whenever new telemetry data arrives, the graph datasets are updated, allowing operators to monitor mission trends throughout the simulation.

4. Telemetry Simulation Engine

Since this project is simulation-based, telemetry packets are generated internally without requiring external hardware.

The telemetry engine is implemented using JavaScript's `setInterval()` function, which executes once every second while telemetry is active.

Each generated packet contains simulated values for:

- Altitude
- Temperature
- Pressure
- Battery Voltage
- Descent Rate
- Roll
- Pitch
- Yaw

Random values within realistic operating ranges are generated to imitate actual CanSat sensor behaviour. After generation, every telemetry packet is stored inside the telemetry array and forwarded to the dashboard update function, graph update function, and error monitoring system.

This simulation accurately demonstrates how a real Ground Control Software processes incoming telemetry packets from an onboard flight computer.

5. Dashboard Update Logic

The dashboard update module is responsible for displaying the latest telemetry values on the operator interface.

Whenever a new telemetry packet is generated, the function updates all numerical fields including:

- Altitude
- Pressure
- Temperature
- Battery Voltage
- Descent Rate
- Roll
- Pitch
- Yaw

The Roll and Pitch values are also used to animate the orientation indicator, providing a visual representation of the CanSat's attitude.

In addition, every received telemetry packet is inserted into the packet log table. This allows mission operators to review previous telemetry packets while the simulation is running.

The dashboard update function ensures that all displayed values remain synchronized with the most recent telemetry packet.

6. Live Graph Update Mechanism

The graph update function receives each telemetry packet and appends its values to the corresponding Chart.js datasets.

The update process includes:

- Adding the current timestamp
- Updating Altitude data
- Updating Temperature data
- Updating Pressure data
- Updating Descent Rate data
- Updating Battery Voltage data

To maintain application performance, only the latest telemetry samples are retained while older values are automatically removed.

Finally, each Chart.js object is refreshed using the `update()` method, producing smooth real-time graphs that continuously display mission behaviour.

7. Error Code Generation System

The Ground Control Software implements a four-digit aerospace error code system to provide rapid mission fault identification.

Each digit represents a different mission condition:

Digit	Mission Condition
First	Descent Rate Safety
Second	GPS Availability
Third	Payload Separation Status
Fourth	Emergency Parachute Status

The software continuously evaluates telemetry parameters against predefined conditions.

Examples include:

- Safe descent rate (8–10 m/s)
- GPS availability
- Payload separation status
- Emergency parachute activation

Whenever a fault is detected, the corresponding digit changes from **0** to **1**. The complete four-digit code is displayed using color-coded indicators, where green indicates normal operation and red indicates the presence of one or more mission faults.

8. Mission Command Functions

The Mission Control module simulates the execution of critical aerospace commands.

Three command buttons are implemented:

- Manual Payload Separation
- Emergency Parachute Deployment
- Redundant System Activation

When an operator clicks a command button, the application immediately updates the mission command status panel with the corresponding operation.

Although these commands do not communicate with physical hardware, they accurately simulate the behaviour of real aerospace Ground Control Software and demonstrate operator interaction during mission

operations.

9. Data Export (CSV and Graph Images)

The software includes built-in data management features that allow operators to preserve telemetry information for later analysis.

The **CSV Export** function converts every telemetry packet stored in memory into comma-separated values and automatically downloads the file.

The **Graph Export** function captures all five telemetry graphs and saves them as PNG image files.

These exported files can be used for:

- Mission documentation
- Engineering analysis
- Research reports
- Performance evaluation
- Presentation purposes

The export functionality eliminates the need for external software to preserve mission data.

10. GPS Map Integration (Leaflet.js)

The GPS tracking module uses the Leaflet.js mapping library together with OpenStreetMap.

A simulated GPS coordinate is generated periodically, and the marker position is updated automatically on the map.

The software continuously:

- Updates latitude
- Updates longitude
- Moves the GPS marker
- Re-centers the map
- Simulates CanSat movement

This module provides operators with a real-time visualization of payload location and demonstrates how Ground Control Software tracks mission trajectory.

11. Live Camera Integration

The application supports live video streaming using the browser's MediaDevices API.

When the operator presses **Start Camera**, permission is requested to access the webcam. After approval, the live video stream is displayed inside the dashboard.

The **Stop Camera** button terminates the video stream by stopping all active media tracks.

This module simulates onboard mission observation and demonstrates how visual monitoring can be integrated into aerospace Ground Control Software.

12. Summary of Functions

The `main.js` file integrates all major software components into a unified Ground Control Software system.

The implemented functions include:

- Telemetry simulation
- Dashboard updating
- Real-time graph visualization
- Packet logging
- Error code generation
- Mission command execution
- GPS tracking
- Orientation visualization
- Live camera streaming
- CSV export
- Graph export
- Packet reset